

Iman Kahssay (915398409),

Joshua Jose Dias Barreto (925586340)

SriJhanyaa Srinivasan (924159943)

Professor Mathew Bishop

ECS 235B: Computer and Information Security

June 8th, 2026

Comparative Analysis of Confidentiality Models in Computer Security

I. Introduction:

Confidentiality is a central goal in computer security, and different access control models define and enforce confidentiality in different ways. This project conducts a comparative literature survey of three confidentiality-focused security models: the Lattice-Based Access Control Model, the Chinese Wall Model, and the Noninterference Confidentiality Model. These models were selected because they each address a different confidentiality problem beyond the Bell-LaPadula Model. Bell-LaPadula provides an important foundation for multilevel security through rules such as “no read-up” and “no write-down,” but later models extend, refine, or reinterpret confidentiality enforcement in more specialized contexts. This report analyzes each model in terms of the problem it addresses, its strengths and limitations, and its appropriate use cases. The Biba Model will also be briefly discussed, not as a primary confidentiality model, but as an important integrity-focused model that also uses a lattice-based structure.

To complement the literature review, this project will also connect the theoretical models to practical behavior by including small test scripts that model subjects (users or processes), objects (files), actions (read or write), and security labels or access categories. Using a unified decision function (such as: “can_access(subject, obj, action, model)”), the scripts will simulate how each confidentiality model evaluates access requests. By comparing the outputs across each model, this project will illustrate how the models differ in enforcing confidentiality, as well as use the results to support the paper’s broader analysis.

II. Literature Review

A. Lattice-Based Access Control Model

The Lattice-Based Access Control (LBAC) model uses ordered security labels to determine whether a subject can access an object, which makes it especially useful for systems that control how information flows across different security levels, roles, or categories. The main problem LBAC addresses is how to prevent unauthorized information flow in systems where users and resources are not all at the same level of privilege. Rather than treating access permissions as isolated rules, LBAC organizes them within a multilevel security structure, where “LBAC is concerned with enforcing one-directional information flow in a lattice of security labels” [1]. This connects LBAC to the Bell-LaPadula confidentiality model, since both use ordered security levels to control information flow. In a confidentiality-focused lattice, information can move “from low to high but not vice versa” [1]. This means that information is allowed to flow upward to higher-security subjects, but can’t move downward to lower-security subjects. This makes LBAC especially appropriate for environments where information must be carefully separated by clearance level, sensitivity, department, role, or conflict-of-interest category.

One major strength of LBAC is its flexibility. Although it is closely related to Bell-LaPadula, LBAC is more general because the lattice can be configured for “confidentiality, integrity, confidentiality and integrity together, or for aggregation policies such as Chinese Walls” [1]. This broader structure helps explain its relationship to both Bell-LaPadula and Biba: all three use ordered labels and lattice-based rules to control information flow, but they enforce different goals. Bell-LaPadula protects confidentiality by preventing information from flowing downward through “no read-up” and “no write-down” rules, while Biba protects integrity by preventing lower-integrity information from corrupting higher-integrity objects through “no read-down” and “no write-up” rules. When used alongside Bell-LaPadula, Biba can also have minor side effects on confidentiality because its integrity restrictions further limit how subjects and objects interact across security levels. Biba does not directly protect secrecy, but by restricting certain reads and writes for integrity reasons, it can reduce potential channels through which information might move between levels. As a result, these models together show how LBAC can support stricter flow-control systems that protect confidentiality while also limiting integrity contamination.

Recent applications show that LBAC can be adapted to practical access-control problems, but they also reveal its limitations. In healthcare, LBAC can protect sensitive electronic health records by modeling access through “subjects,” “objects,” and “transactions,” where transactions include actions such as “read, write, and execute” [2]. This structure will be useful for the test-script portion of this report, because it describes how access requests can be simulated between users, protected files, and allowed actions.

In contrast, LBAC depends heavily on correctly assigned labels and lattice relationships; if labels are too broad, users may receive too much access, while overly restrictive labels can block legitimate work. Traditional LBAC may also need extensions in systems where access depends on changing context, relationships, or user attributes. For example, one paper adapts LBAC to online privacy by “extract[ing] users’ attributes to form the lattice element” before lattice elements during access decisions [3], while another paper on real-time policy updates shows that outdated policies can create security gaps when “the policy is changed while it is in effect and this change [is not] enforced immediately [by the system]” [4].

Overall, LBAC is strongest in structured environments such as classified systems, healthcare records, databases, and dynamic policy systems where information-flow rules must be clear, enforceable, and testable, but it can become difficult to manage when labels, relationships, or policies change frequently.

B. Chinese Wall Model

Brewer and Nash developed the Chinese Wall Security Model to avoid conflicts of interest in businesses including banks, law firms, and consulting firms [5]. The main problem the Chinese Wall Model addresses is how to protect confidentiality when users may have legitimate access to sensitive business data, but should not be allowed to combine information from competing organizations. Unlike military-oriented confidentiality models such as Bell-LaPadula, which rely on predefined security classifications, the Chinese Wall Model makes access decisions based on a user’s past access history. Information is organized into “company datasets” and more general “conflict-of-interest (COI) classes,” where rival businesses are grouped together [5]. Once a user accesses confidential information from one company in a COI class, the model prevents that user from accessing confidential information from competing companies in

the same class. This makes the model dynamic and context-sensitive because access is not determined only by who the user is, but by what information the user has already seen.

One major strength of the Chinese Wall Model is that it captures confidentiality problems that fixed security labels do not fully address. In business settings, the main risk is not always that information flows from a high level to a low level; instead, the risk could be that one person gains knowledge from multiple competing clients and uses that knowledge inappropriately. This makes the Chinese Wall Model especially useful in environments where confidentiality depends on client separation, conflict-of-interest restrictions, and preventing insider exploitation. The model can also be combined with Role-Based Access Control (RBAC), allowing organizations to enforce both role-based permissions and conflict-of-interest limitations [6]. This broader structure helps explain why the Chinese Wall Model is often more suitable than Bell-LaPadula for commercial systems: Bell-LaPadula uses fixed hierarchical levels and rules such as “no read up” and “no write down” to control information flow, whereas the Chinese Wall Model bases access decisions on a subject’s prior access history to prevent conflicts of interest.

Recent research shows that the Chinese Wall Model can also be adapted beyond traditional business environments. It can be applied to cloud environments, databases, processes, and distributed systems where multiple organizations may share infrastructure but still require strong confidentiality boundaries. For example, a study on workflow security shows that inter-organizational systems need safeguards against users accessing rival organizational data during shared processes [7]. Similarly, another paper on database security shows that conflict-of-interest policies can help separate sensitive records in shared systems [8]. These applications are particularly crucial in cloud computing environments, where strict confidentiality boundaries between datasets are required even though numerous firms may share infrastructure. Collectively, these studies show that the Chinese Wall Model can protect confidentiality in modern distributed and data-intensive environments, and not only in conventional commercial systems.

However, the Chinese Wall Model also has important limitations. Because access decisions depend on prior access history, the system must accurately record what each user has accessed over time. In large-scale systems, this can become difficult to manage, especially when

many users, companies, datasets, and COI classes are involved. If COI classes are defined too broadly, the model may block legitimate work; if they are defined too narrowly, the system may fail to prevent risky access. As a result, the Chinese Wall Model is strongest in commercial environments where conflict-of-interest relationships are clear and must be enforced dynamically, but it can become difficult to maintain when access histories or organizational relationships change frequently. Despite these limitations, the Chinese Wall Model remains a useful framework for maintaining confidentiality in business systems where competitive data exposure and insider conflicts are significant issues [5][6].

C. Noninterference Confidentiality Model

The Noninterference Confidentiality Model was introduced by [9], and is a strict multilevel security policy model that is designed to address a fundamental limitation of rule-based models such as Bell-LaPadula: they can prevent direct unauthorized reads and writes, but they do not always prevent indirect information leakage through covert channels. The main problem Noninterference addresses is how to ensure that high-security activity cannot influence what low-security users are able to observe. The model assumes that a system can be represented as a deterministic state machine divided into security domains, where inputs and outputs are classified as HIGH (sensitive) or LOW (observable by lower-security users) [9]. Under this model, a system satisfies the noninterference property if and only if the same low-level inputs always produce the same low-level outputs, regardless of what high-level inputs occurred [9]. In simpler terms, high-security actions must have no observable effect on low-security users.

This provides a stronger confidentiality guarantee than Bell-LaPadula because it addresses indirect and covert flows, not just direct read and write access. Although Bell-LaPadula is concerned with direct read and write access between security levels via its simple security and star property rules, it does not take into consideration the indirect flows, where a high-security process can influence a low-security observable output using shared resources, timing behavior, or storage channels without violating a read or write rule [11]. Noninterference reduces this gap by treating confidentiality as a system-wide behavioral property instead of a per-request access decision. This means that even indirect or implicit flows that satisfy Bell-LaPadula's rules are caught if they alter the low-level view of the system [9][11].

As a result, Noninterference is especially important in adversarial settings where indirect channels are a serious threat.

One of the model's main strengths is its focus on policy objectives rather than enforcement mechanisms, and this makes it applicable beyond access control to the formal verification of operating systems and security-critical software [9]. Rushby extended the model's practical use by introducing intransitive noninterference, where information may flow from a high domain to a low domain only through an approved trusted downgrader [10]. This is more realistic for systems that require controlled declassification, as it allows limited, authorized information release while still preserving strict control over unauthorized flows [10]. Noninterference has also been used as a verification target in high-assurance systems. For example, [12] provides the first complete and machine-checked proof of information flow security for the seL4 microkernel, which is applied directly to its C implementation. This shows that the kernel allows no information flows other than those permitted by its access control policy.

However, Noninterference is also difficult to apply in practice. Because the original formulation assumes deterministic systems, it is harder to use with nondeterministic or randomized systems, including encrypted communication protocols where probabilistic behavior is expected [9]. [11] explained how a model can be overly restrictive because certain legitimate multilevel security operations, such as releasing statistical summaries of sensitive datasets, may be rejected if low-level outputs depend in any way on high-level information. Hence, this model is not well suited for systems that require controlled or partial information release. In addition, the original purge-based definition can struggle with intransitive policies, and the seL4 case described by [12] shows that even small design choices, such as scheduler behavior, can introduce possible violations. This would make full compliance demanding to achieve and verify in practice. Despite these limitations, Noninterference remains a theoretical standard for systems where preventing indirect leakage is non-negotiable, such as separation kernels, military-grade systems, cryptographic protocol verification, and language-based information flow security [13].

III. Model Comparisons

LBAC and the Chinese Wall Model both address confidentiality, but they differ in the kind of confidentiality problem they are best suited to solve. **LBAC is stronger when a system needs a consistent, formally structured access-control policy that can be evaluated through labels, dominance relationships, and clear information-flow rules.** This makes it easier to reason about and test because access decisions follow from the lattice structure rather than from each user's individual history. However, that same structure can make LBAC less responsive to business-specific situations where the meaning of "allowed access" changes based on what a user has already seen. In other words, LBAC is powerful for predictable security hierarchies, but it may need additional policy layers to capture conflicts of interest, client separation, or relationship-based restrictions. The **Chinese Wall Model fills that gap by making confidentiality dependent on context and prior access**, which is useful when the risk is not simply that information flows from high to low, but that one person may combine sensitive knowledge from competing organizations. Its strength is that it can adapt access decisions over time as a user interacts with different datasets, making it more realistic for commercial environments. However, this flexibility also makes the Chinese Wall Model harder to manage than LBAC because the system must continuously track each user's access history and carefully maintain conflict-of-interest groupings. If these histories or groupings are inaccurate, the model may either block legitimate work or fail to prevent risky access. Therefore, LBAC is generally stronger as a broad, formal framework for enforcing information-flow rules, while the Chinese Wall Model is stronger as a specialized confidentiality model for preventing competitive misuse of sensitive business information.

The Chinese Wall Model and the Noninterference Model both protect confidentiality, but they are designed for very different security problems. **The Chinese Wall Model treats conflicts of interest as the main confidentiality risk, especially in commercial settings where users may work with sensitive data from competing companies. Its main strength is that access decisions are dynamic and context-sensitive: a user's permissions change depending on what confidential information they have already accessed.** This makes the model useful in industries such as banking, consulting, healthcare, and cloud computing, where users might need legitimate access to sensitive data, but must be prevented from combining information from conflicting datasets. However, this flexibility also creates management challenges, because large

systems must accurately track user access histories and define conflict-of-interest (COI) classes correctly. The Noninterference model, by contrast, provides a stricter confidentiality guarantee by requiring that high-security actions have no observable effect on low-security users. Its **robust mathematical assurance of confidentiality is its main advantage, which makes it better suited for high-assurance environments** such as secure operating systems, military systems, separation kernels, and formally certified environments, **where even indirect information leaks through shared resources, timing, or system behavior must be prevented.** However, because the Noninterference model restricts even controlled or partial information sharing, it can sometimes be viewed as too rigid for real-world enterprise systems. Overall, the **Chinese Wall Model is more practical for commercial environments that require flexible but history-based confidentiality enforcement, while Noninterference is more appropriate for systems that require rigorous isolation and strong guarantees against indirect leakage.**

The Noninterference Model and the Lattice-Based Access Control Model (LBAC) both address confidentiality, but they do so at different levels of enforcement. The LBAC model organizes security labels into a lattice structure and uses dominance relationships to determine whether a subject can access an object. This makes it effective for systems with well-defined classification hierarchies, such as government document management, healthcare record systems, and corporate data tiering, where access decisions need to be clear, deployable, and auditable. However, LBAC mainly controls explicit access requests and may not detect indirect or covert information flows that occur through shared resources, timing behavior, or other observable system effects. **The Noninterference model addresses this limitation by enforcing a stronger system-wide guarantee, where high-level activity should have no observable effect on low-level users.** Because of this, **Noninterference provides stronger confidentiality assurance than LBAC, because it catches covert and indirect flows that LBAC's label-based rules would not catch.** However, it is also more restrictive and harder to implement in practical systems. **LBAC is better suited for operational systems that need a deployable, auditable access control policy with well-understood performance characteristics.** Noninterference is better for high-assurance environments where formal verification is a requirement, and even indirect leakage is unacceptable, such as separation kernels, military avionics software, and secure microkernel design. In conclusion, LBAC answers the question of who is allowed to access what, while Noninterference answers the stricter question of whether a low-level observer

can detect any effect of high-level activity, which is useful in adversarial settings where indirect channels are a threat.

IV. Conclusion/Analysis

The Python test script, “[LBAC_test.py](#)”, extends the discussion of LBAC by translating the model’s abstract lattice structure into a concrete computer-file access scenario. It models a file system where users and files are assigned ordered security labels: Guest, Standard User, Power User, and Admin. In the script, users act as subjects, files act as protected objects, and read and write operations represent access transactions. The main access-control decision is handled through a dominance function, which checks whether one security level is greater than or equal to another. Structurally, the script creates test users, protected files, and actions, then runs every possible user-file-action combination through both the practical LBAC policy and the Bell-LaPadula-style confidentiality policy. The output contributes to the analysis by making the model’s behavior visible: each test case produces an “ALLOW” or “DENY” result, showing how small policy changes can produce different access-control outcomes even when the same lattice is used. **This demonstrates why LBAC is useful for structured systems: access decisions are based on predictable relationships between labels rather than individual case-by-case permissions. The script also shows LBAC’s flexibility by testing two policy goals on the same lattice.** The practical LBAC policy allows higher-level users to read and write lower-level files, which fits normal administrative systems where an Admin might need to manage public, user, developer, and system files. The Bell-LaPadula configuration, however, denies high-level users the ability to write downward, showing how the same lattice can be tightened to prevent confidentiality leaks.

This behavior supports the argument that **LBAC is strongest in structured environments where access can be clearly organized by privilege level, such as operating systems, company file systems, databases, and healthcare record systems.** In these settings, users and resources can be clearly labeled and access decisions can be tested through predictable dominance relationships.

The Python test script, “[Chinese_Wall_test.py](#)”, extends the discussion of the Chinese Wall Model by translating its history-based confidentiality rules into a concrete business

access-control scenario. The script includes conflict-of-interest classes such as “Technology” and “Healthcare”, company files as protected objects classified as either “Public” or “Confidential”, users as subjects, and read/write requests as access transactions. The core principle of the model is that once a user accesses confidential information from one company within a conflict class, they cannot access confidential information from competing companies in the same conflict class. To test this behavior, the script processes a sequence of read and write requests while maintaining a record of each user’s previous confidential accesses. This structure is important because the Chinese Wall Model does not only ask whether a user has enough privilege to access a file; it asks whether granting access would create a conflict of interest based on what that user has already seen. The output makes this behavior visible by returning “ALLOW” or “DENY” for each request depending on the user’s access history, the file’s company, and the file’s conflict-of-interest class.

The script’s behavior supports the argument that the Chinese Wall Model is strongest in commercial environments, where confidentiality depends on preventing users from combining sensitive information from competing organizations. For example, once Alice accesses Google’s confidential data, she is later denied access to Microsoft’s confidential data because both companies belong to the same “Technology” conflict-of-interest class. However, she can still access Pfizer’s confidential data because it belongs to a different conflict class, and public files remain accessible to everyone. **This demonstrates the model’s main strength: its access decisions adapt over time based on prior activity rather than following a fixed hierarchy.** The mixture of “ALLOW” and “DENY” outcomes demonstrates that the Chinese Wall model dynamically restricts access to competing companies while still permitting access to the same company, different conflict classes, and public information. **Unlike access-decision models such as LBAC or Bell-LaPadula, the Chinese Wall model protects confidentiality by preventing conflicts of interest based on prior access history.** Therefore, the script shows why the Chinese Wall Model is better suited for consulting firms, banks, healthcare organizations, cloud platforms, and other business settings where users may need legitimate access to sensitive data while still being blocked from conflicting datasets.

The Python test script, “[Noninterference_test.py](#)”, extends the discussion of Noninterference by testing confidentiality as a system-wide behavioral property rather than as a

single access-control decision. The script models a four-level security hierarchy of files and users (Guest, Standard User, Power User, and Admin). It then evaluates whether the noninterference property holds under a secure baseline and three flawed implementations. The central idea being tested is that a LOW-level observer's view of the system should remain unchanged regardless of whether a HIGH-level user performs an action. To evaluate this, the test script runs two parallel system executions: one where a HIGH user writes to the system and one where the HIGH user does not. The LOW observer's view is then compared across both executions. If the observations differ, noninterference is broken (and the program returns "BROKEN") because high-level activity has produced a low-level observable effect, meaning information has leaked across security levels even though no direct access-control rule was violated. This structure captures the central idea of Noninterference: high-security activity should not produce any observable effect for low-security users.

The script's behavior supports the argument that **Noninterference is strongest in high-assurance environments where even indirect leakage is unacceptable**. The secure baseline shows the intended behavior: every test returns "HOLDS" because writes only propagate upward, reads are level-gated, and HIGH activity remains invisible to LOW observers. **The flawed implementations then show why Noninterference provides a stricter guarantee than LBAC or Bell-LaPadula-style access control: even when direct file access is blocked, information can still leak through different mechanisms, and not every combination of users and files produces a violation.** However, the results also show that violations are not automatic; whether a case returns "HOLDS" or "BROKEN" depends on the relationship between the HIGH writer, the LOW observer, the target file, and the specific system behavior being exposed.

In the first flaw (A), "HOLDS" appears whenever the HIGH writer is denied the write, because if the write never occurs, there is no downward effect for the LOW observer to detect. Similarly, files requiring Admin access hold for all lower observers, since no non-Admin HIGH writer can touch them. In the second flaw (B), the pattern is almost reversed: "BROKEN" only appears when a denied write attempt from a HIGH user changes a shared counter, allowing the LOW observer to detect that HIGH-level activity occurred. Successful writes do not change that counter, so those cases continue to hold, where the HIGH user writing within their own privilege

range produces no observable signal. In the third flaw (C), “HOLDS” appears for “system_config.conf” when observed by users below Admin because those observers should not be able to access the file’s size metadata, leaving them with no visible channel through which to detect the write. Together, the mixed “HOLDS” and “BROKEN” results demonstrate that violations depend on the specific intersection between the writer’s access, the observer’s access, and which aspects of the system state are shared. **This serves as a useful reminder that security analysis must account for every observable part of the system, not just file contents.** Therefore, the script shows why Noninterference is best suited for secure operating systems, separation kernels, military systems, cryptographic protocols, and formally verified software where confidentiality requires protection against both direct and indirect information flows.

The combined test results show that the three models differ not only in their access rules, but also in what they treat as evidence of a confidentiality violation. In “[LBAC_test.py](#)”, a violation is understood as an unauthorized access decision: a user either dominates the file’s required label or does not. This produces clean and predictable “ALLOW” and “DENY” results, which is why LBAC works well when confidentiality can be reduced to stable labels and dominance relationships. However, the predictability of the LBAC output also reveals its limitation: the script has no memory of prior access and does not evaluate whether an allowed action creates a later risk. By contrast, “[Chinese Wall test.py](#)” shows that confidentiality can depend on sequence and context. A request that is safe at one point may become unsafe later because the user’s access history changes the meaning of the request. **This demonstrates that confidentiality is not always a static relationship between one subject and one object; in commercial systems, it may depend on whether access would allow a user to combine sensitive knowledge across competing organizations.**

The “[Noninterference_test.py](#)” results push this analysis even further by showing that confidentiality can fail even when access-control decisions appear correct. In the flawed implementations, some cases return “BROKEN” not because a LOW user directly reads a HIGH file, but because HIGH activity changes something the LOW observer can detect, such as a shared counter or metadata. At the same time, the mixed “HOLDS” and “BROKEN” results show that indirect leakage is not automatic; it depends on whether the HIGH action changes a

part of the system state that is visible to the LOW observer. This is the key difference between Noninterference and the other two models. LBAC and Chinese Wall mainly evaluate whether a requested access should be allowed, while Noninterference evaluates whether the system's observable behavior changes across executions. As a result, **Noninterference exposes a deeper kind of confidentiality risk: information can leak through side effects even when the visible file permissions appear to be enforced correctly.** Moreover, these results show that **each model prevents information leak in a way that the previous one misses, and collectively, these three scripts show that confidentiality is not a single property, but a spectrum of guarantees that is each suited to a different threat environment.**

Taken together, the three models show that confidentiality is not a single access-control problem, but a set of different guarantees depending on what kind of leakage the system needs to prevent. The LBAC model is strongest when access can be organized through stable labels and dominance relationships, the Chinese Wall model is strongest when confidentiality depends on conflict-of-interest history, and the Noninterference model is strongest when even covert or indirect information leakage must be ruled out. The test scripts reinforce these distinctions by showing that different outputs do not simply come from different implementation choices; they come from different definitions of what counts as a confidentiality violation. Therefore, the best model depends on whether the system prioritizes structured privilege enforcement, history-sensitive conflict prevention, or high-assurance isolation.

V. RESOURCES

To view and run the test scripts mentioned in the report, please follow the link to our GitHub repository [here](#).

References

- [1] R. S. Sandhu, “Role Hierarchies and Constraints for Lattice-Based Access Controls,” in *Computer Security — ESORICS 96, Lecture Notes in Computer Science*, vol. 1146, Berlin, Heidelberg: Springer, 1996, pp. 65–79, doi: 10.1007/3-540-61770-1_28. URL: https://doi.org/10.1007/3-540-61770-1_28
- [2] T. Haritha and A. Anitha, “Multi-Level Security in Healthcare by Integrating Lattice-Based Access Control and Blockchain-Based Smart Contracts System,” *IEEE Access*, vol. 11, pp. 114322–114340, 2023, doi: 10.1109/ACCESS.2023.3324740. URL: <https://doi.org/10.1109/ACCESS.2023.3324740>
- [3] Y. Zhang, K. Chen, Y. Liu, and Y. Lian, “A Lattice-Based Access Control Model for Social Networks,” in *2016 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC)*, 2016, pp. 54–61, doi: 10.1109/CyberC.2016.19. url: <https://doi.org/10.1109/CyberC.2016.19>
- [4] T. Xin and I. Ray, “A Lattice-Based Approach for Updating Access Control Policies in Real-Time,” *Information Systems*, vol. 32, no. 5, pp. 755–772, 2007, doi: 10.1016/j.is.2006.06.002. url: <https://doi.org/10.1016/j.is.2006.06.002>
- [5] D. F. C. Brewer and M. J. Nash, “The Chinese Wall Security Policy,” *Proceedings of the IEEE Symposium on Security and Privacy*, 1989
- [6] R. S. Sandhu, “Role-Based Access Control Models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [7] M. H. Kang, J. S. Park, and J. N. Froscher, “Access Control Mechanisms for Inter-Organizational Workflow Security,” *Proceedings of the ACM Symposium on Access Control Models and Technologies*, 2001.

[8] E. Bertino and R. Sandhu, "Database Security—Concepts, Approaches, and Challenges," IEEE Transactions on Dependable and Secure Computing, vol. 2, no. 1, pp. 2–19, 2005

[9] J. A. Goguen and J. Meseguer, "Security Policies and Security Models," in Oakland, CA, USA, 1982, pp. 11-11, doi: 10.1109/SP.1982.10014.URL:
<https://doi.ieeecomputersociety.org/10.1109/SP.1982.10014>

[10] Rushby, J. (1992, December). Noninterference, Transitivity, and Channel-Control Security Policies. Technical Report SRI-CSL-92-2, Computer Science Laboratory, SRI International, Menlo Park, CA.

[11] McLean, John D. "Security models and information flow." Proceedings. 1990 IEEE Computer Society Symposium on Research in Security and Privacy (1990): 180-187.

[12] Toby Murray, Daniel Matichuk, Matthew Brassil, Peter Gammie, Timothy Bourke, Sean Seefried, Corey Lewis, Xin Gao, and Gerwin Klein. 2013. SeL4: From General Purpose to a Proof of Information Flow Enforcement. In Proceedings of the 2013 IEEE Symposium on Security and Privacy (SP '13). IEEE Computer Society, USA, 415–429.
<https://doi.org/10.1109/SP.2013.35>

[13] Sabelfeld, Andrei and Andrew C. Myers. "Language-based information-flow security." IEEE J. Sel. Areas Commun. 21 (2003): 5-19.